Python Programming Notes

Beginner notes covering 8 units, for college exam preparation.

Unit 1: Introduction to Python

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and released in 1991. It is popular because the syntax is simple and readable, and it is used in web development, data science, machine learning, automation, and scripting.

Features of Python: easy to learn, interpreted (code runs line by line), dynamically typed (no need to declare variable types), object-oriented, portable across Windows, Linux and macOS, and supported by a very large standard library.

A Python program is saved with the extension .py and run using the command python filename.py. The print() function displays output on the screen. Comments start with the # symbol and are ignored by the interpreter.

Variables store data. A variable is created when a value is assigned to it, for example age = 20. Variable names must start with a letter or underscore and cannot be a keyword such as if, for, or while. Python is case sensitive, so Name and name are different variables.

Example:

```
name = 'Asha'
age = 20
print('Hello', name)
print('Age:', age)
```

Unit 2: Data Types and Operators

Python has built-in data types: int (whole numbers), float (decimal numbers), str (text), bool (True or False), list, tuple, set, and dict. The type() function returns the type of a value.

Arithmetic operators are + (add), - (subtract), * (multiply), / (divide), // (floor division), % (remainder), and ** (power). Comparison operators are ==, !=, >, <, >=, <=. Logical operators are and, or, not.

Type conversion changes one type to another: int('5') gives 5, str(10) gives '10', float(3) gives 3.0. The input() function reads text from the user and always returns a string, so convert it with int() if a number is needed.

Strings are sequences of characters. They support indexing (name[0] is the first character), slicing (name[0:3]), and methods such as upper(), lower(), strip(), replace(), split(), and len(). Strings are immutable, which means they cannot be changed after creation.

Example:

```
x = 10
y = 3
print(x + y, x // y, x % y, x ** y)
text = 'python'
```

```
print(text.upper(), text[0:3], len(text))
```

Unit 3: Control Statements and Loops

Conditional statements let a program make decisions. The if statement runs a block when a condition is True. Use elif for extra conditions and else for the default case. Python uses indentation (4 spaces) to mark a block, not curly braces.

A for loop repeats a block for each item in a sequence such as a list, string, or range(). The range(n) function produces numbers from 0 to n-1. A while loop repeats as long as its condition stays True, so the condition must eventually become False to avoid an infinite loop.

The break statement exits a loop immediately. The continue statement skips the rest of the current iteration and moves to the next one. The pass statement does nothing and is used as a placeholder.

Example:

```
marks = 72
if marks >= 90:
    print('Grade A')
elif marks >= 60:
    print('Grade B')
else:
    print('Grade C')

for i in range(1, 6):
    print(i)
```

Unit 4: Lists, Tuples, Sets and Dictionaries

A list is an ordered, mutable collection written in square brackets, for example [1, 2, 3]. Common methods are append() to add an item, insert() to add at a position, remove() and pop() to delete, sort() to arrange, and reverse(). Lists can hold different data types.

A tuple is an ordered, immutable collection written in round brackets, for example (1, 2, 3). Once created, its items cannot be changed. Tuples are faster than lists and are used for fixed data such as coordinates.

A set is an unordered collection of unique items written in curly braces. It automatically removes duplicates and supports union, intersection, and difference operations.

A dictionary stores data as key-value pairs, for example {'name': 'Asha', 'age': 20}. Values are accessed by key. Useful methods are keys(), values(), items(), get(), and update(). Keys must be unique and immutable.

List comprehension is a short way to create a list: squares = [n * n for n in range(5)] gives [0, 1, 4, 9, 16].

Example:

```
fruits = ['apple', 'banana']
fruits.append('mango')
student = {'name': 'Asha', 'age': 20}
```

```
print(student['name'])
print([n * n for n in range(5)])
```

Unit 5: Functions and Modules

A function is a reusable block of code defined with the def keyword. It can take parameters and return a value using the return statement. Functions avoid repeating code and make programs easier to read and test.

Parameters can have default values, for example def greet(name='friend'). Keyword arguments let you pass values by name. *args collects extra positional arguments into a tuple, and **kwargs collects extra keyword arguments into a dictionary.

A lambda function is a small anonymous function written in one line, for example square = lambda x: x * x. Variables created inside a function are local, while variables created outside are global.

A module is a Python file that contains functions and variables. Use the import statement to use it, for example import math or from math import sqrt. A package is a folder of modules. Libraries are installed with pip, for example pip install pandas.

Example:

```
def add(a, b=0):
    return a + b

print(add(5, 3))
square = lambda x: x * x
print(square(4))
```

Unit 6: Object Oriented Programming

Object Oriented Programming (OOP) organizes code using classes and objects. A class is a blueprint, and an object is an instance of the class. The __init__ method is the constructor and runs automatically when an object is created. The self keyword refers to the current object.

The four pillars of OOP are encapsulation (hiding data inside a class), inheritance (a child class reuses the code of a parent class), polymorphism (the same method name behaves differently in different classes), and abstraction (showing only the necessary details).

Inheritance is written as class Child(Parent). The super() function calls the parent class constructor or methods. Method overriding means a child class defines a method with the same name as the parent to change its behavior.

Example:

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def show(self):
        print(self.name, self.marks)

s = Student('Asha', 85)
s.show()
```

Unit 7: File Handling and Exception Handling

File handling lets a program read and write files. Use open(filename, mode) where mode is 'r' for read, 'w' for write (overwrites), 'a' for append, and 'r+' for read and write. The with statement closes the file automatically, so it is the recommended way.

Common file methods are read(), readline(), readlines(), and write(). Reading a CSV file is easy with the pandas function read_csv().

An exception is an error that happens while the program runs, such as dividing by zero or opening a missing file. Use try and except to handle it so the program does not crash. The finally block always runs. The raise keyword creates an exception on purpose.

Common exceptions: ZeroDivisionError, ValueError (wrong value, for example int('abc')), TypeError, IndexError (list index out of range), KeyError (missing dictionary key), and FileNotFoundError.

Example:

```
try:
    with open('notes.txt', 'r') as f:
        print(f.read())
except FileNotFoundError:
    print('File not found')
finally:
    print('Done')
```

Unit 8: Python Libraries for Data Science

NumPy provides fast arrays and mathematical operations. Pandas provides the DataFrame, a table of rows and columns used to clean and analyze data. Matplotlib draws charts and graphs. Scikit-learn offers machine learning algorithms such as regression, classification, and clustering.

In Pandas, read_csv() loads a file, head() shows the first rows, describe() gives statistics, dropna() removes missing values, and groupby() groups rows to calculate totals or averages.

Streamlit is a Python library that turns scripts into web apps with very little code. Requests is a library for calling web APIs. Virtual environments (python -m venv .venv) keep the packages of each project separate.

Example:

```
import pandas as pd

df = pd.read_csv('data.csv')
print(df.head())
print(df.groupby('category')['amount'].sum())
```